

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANSWER SCRIPT

CO1: Apply AI basics such as intelligent agents and

uninformed search methods to solve problems effectively. (BL3)

Subject	Artificial Intelligence
Course Code	CSA 1712
Name	Vignesh Kumar K
Register Number	192512375

Declaration

I, Vignesh Kumar K (Reg No: 192512375), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Question 1 — The Water Jug Problem

Problem: Given a 4-gallon jug and a 3-gallon jug (both unmarked) and an unlimited water supply, obtain exactly 2 gallons of water in the 4-gallon jug.

1. Problem Formulation as a State-Space Search

State representation: A state is an ordered pair (x, y) , where x = amount of water currently in the 4-gallon jug ($0 \leq x \leq 4$) and y = amount of water currently in the 3-gallon jug ($0 \leq y \leq 3$).

Initial state: $(0, 0)$ — both jugs empty.

Goal state: $(2, y)$ for any value of y — the 4-gallon jug contains exactly 2 gallons.

2. Production Rules (Operators)

Rule	Description	Effect	Precondition
1	Fill the 4-gallon jug from the pump	$(x, y) \rightarrow (4, y)$	$x < 4$
2	Fill the 3-gallon jug from the pump	$(x, y) \rightarrow (x, 3)$	$y < 3$
3	Empty the 4-gallon jug on the ground	$(x, y) \rightarrow (0, y)$	$x > 0$
4	Empty the 3-gallon jug on the ground	$(x, y) \rightarrow (x, 0)$	$y > 0$
5	Pour water from 3-gal jug into 4-gal jug until 4-gal jug is full	$(x, y) \rightarrow (4, y - (4 - x))$	$x + y \geq 4, y > 0$
6	Pour water from 4-gal jug into 3-gal jug until 3-gal jug is full	$(x, y) \rightarrow (x - (3 - y), 3)$	$x + y \geq 3, x > 0$
7	Pour all water from 3-gal jug into 4-gal jug	$(x, y) \rightarrow (x + y, 0)$	$x + y \leq 4, y > 0$
8	Pour all water from 4-gal jug into 3-gal jug	$(x, y) \rightarrow (0, x + y)$	$x + y \leq 3, x > 0$

3. Solution Path (Search Trace)

Step	Current State	Action Applied	Resulting State
1	$(0, 0)$	Fill the 3-gallon jug	$(0, 3)$
2	$(0, 3)$	Pour 3-gallon jug into 4-gallon jug	$(3, 0)$
3	$(3, 0)$	Fill the 3-gallon jug	$(3, 3)$
4	$(3, 3)$	Pour 3-gallon jug into 4-gallon jug until full	$(4, 2)$
5	$(4, 2)$	Empty the 4-gallon jug	$(0, 2)$
6	$(0, 2)$	Pour 3-gallon jug into 4-gallon jug	$(2, 0)$ ✓ GOAL

4. Interpretation of Result

Starting from (0, 0), the goal state (2, 0) is reached in exactly 6 moves. At this point the 4-gallon jug holds exactly **2 gallons** of water, and the 3-gallon jug is empty, satisfying the goal test. This shows that the water-jug puzzle can be systematically solved using state-space search, where each node is a jug-content pair and each edge is a legal pouring/filling/emptying operation.

Question 2 — Mars Rover as an Intelligent Agent

(a) Percepts of the Agent

- Visual images and video captured by onboard stereo/panoramic cameras
- Spectrometer and chemical-analysis readings of soil and rock samples
- Distance and depth data from LIDAR / obstacle-detection sensors
- Positioning data from onboard navigation systems (relative to landing site)
- Atmospheric data — temperature, pressure, dust levels
- Wheel odometry and tilt/inclination sensor readings
- Battery charge level and solar-panel power input
- Commands and mission updates received from Earth mission control

(b) Characterization of the Operating Environment

Property	Value	Justification
Observability	Partially observable	Cameras and sensors cover only the rover's immediate surroundings, not the entire Martian terrain.
Number of agents	Single-agent	The rover operates largely independently, occasionally coordinating with orbiters/mission control.
Determinism	Stochastic	Wheel slippage, dust, and uneven terrain make the outcome of an action uncertain.
Episodic vs Sequential	Sequential	Current movement and sampling decisions affect the state and options available later.
Static vs Dynamic	Dynamic (semi-dynamic)	Weather and dust storms can change the environment while the rover is deliberating.
Discrete vs Continuous	Continuous	Position, sensor readings, and movement are continuous-valued.
Known vs Unknown	Unknown (becomes known through exploration)	The rover builds a map of Mars terrain as it explores; it is not known in advance.

(c) Actions the Agent Can Take

- Move forward / backward / turn left or right
- Steer around or over an obstacle

- Extend/operate the robotic arm to drill or scoop a soil/rock sample
- Capture photographs / panoramic scans of the terrain
- Run onboard spectrometer / chemical analysis on collected samples
- Transmit collected data and images to Earth via orbiter relay
- Deploy or stow scientific instruments
- Enter low-power/sleep mode to conserve battery

(d) Performance Measure

- Distance safely traversed without damage or getting stuck
- Number and scientific quality of samples successfully analysed
- Accuracy and completeness of data/images transmitted to Earth
- Efficient use of battery power and time
- Number of mission objectives (target sites) reached
- Avoidance of hazards such as steep slopes, rocks, or sand traps

(e) Most Suitable Agent Architecture

Recommended architecture: A **Goal-based / Utility-based agent with a model-based (internal state) core** is most suitable.

Justification:

- Since the Mars environment is only partially observable, the rover must maintain an internal model/state (a map of explored terrain, its own position, and remaining power) — this requires a model-based reflex component for real-time hazard avoidance.
- Because the rover must plan a route to reach specific scientific targets (goal locations), it needs goal-based reasoning rather than simple reflexes.
- Since multiple action sequences can achieve a goal (e.g., different paths to a rock sample) with different costs in time and battery power, a utility function is needed to rank and choose the best sequence of actions — making it, more precisely, a utility-based agent.

Question 3 — Formulating the 8-Queens Problem as a Search Problem

Problem statement: Place 8 queens on an 8×8 chessboard so that no two queens attack each other, i.e., no two queens share the same row, column, or diagonal.

Efficient Incremental Formulation

Component	Description
States	Any arrangement of 0 to 8 queens on the board, with at most one queen placed per column, restricted to the left-most columns, and with no queen already attacking another.
Initial State	An empty chessboard (0 queens placed).
Actions	Add a queen to any square in the left-most empty column such that it is not attacked by any queen already on the board.
Transition Model	Returns the board configuration with the new queen added at the specified square.
Goal Test	8 queens are placed on the board and no two queens attack each other (no shared row, column, or diagonal).
Path Cost	Not significant here — each step has a cost of 1, since only the final configuration matters, not the path taken to reach it.

Why This Formulation Is Efficient

By restricting placement to one queen per column and only in the left-most empty column, illegal states (queens already attacking each other) are eliminated as soon as they occur. This reduces the search space from $C(64, 8) \approx 4.4 \times 10^9$ possible boards (placing 8 queens anywhere on 64 squares) down to about 2,057 states that need to be examined — a dramatic improvement in efficiency.

Relevance of the Given Figure

The figure shows one partial/complete placement of 8 queens where the queen in the first column and the queen in the last column both lie on the same diagonal. Since two queens share a diagonal, this configuration *fails the goal test* and is therefore not a valid solution — it illustrates exactly the kind of conflict the goal test (and the incremental action constraint) is designed to catch and reject.

Question 4 — Problem-Solving Agent for OLA Cab Booking

Scenario: A customer wants to travel from a pickup location to a destination using the OLA app, choosing among cab types (Mini, Micro, Sedan, Shared, Prime, etc.) based on comfort, cost, and time.

Type of Agent

This is best modelled as a Utility-Based Agent (a specialised form of a goal-based agent).

- Goal-based reasoning alone is insufficient here: simply "reaching the destination" can be satisfied by any cab type, so the agent cannot decide which single action sequence to prefer using goal-satisfaction alone.
- Multiple options (Mini, Micro, Sedan, Shared, Prime) all achieve the same goal but differ in cost, comfort, and waiting time — the agent must rank these competing options using a utility (preference) function.

- The agent therefore combines goal-based planning (route to the destination) with a utility function that scores each candidate cab option, then selects the option with the highest expected utility for the customer.

Pseudocode

```
function OLA_CAB_BOOKING_AGENT(pickup, destination, preferences):
    percept <- GET_CURRENT_LOCATION(pickup)
    availableCabs <- GET_AVAILABLE_CAB_TYPES(pickup) // {Mini, Micro, Sedan,
    Shared, Prime, ...}

    for each cabType in availableCabs:
        cost[cabType] <- ESTIMATE_FARE(pickup, destination, cabType)
        eta[cabType] <- ESTIMATE_ARRIVAL_TIME(pickup, cabType)
        comfort[cabType] <- COMFORT_RATING(cabType)
        utility[cabType] <- UTILITY_FUNCTION(cost[cabType], eta[cabType],
        comfort[cabType], preferences)

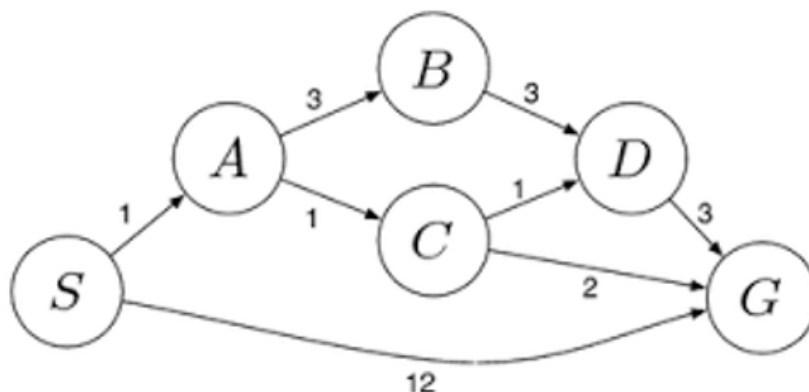
    bestCab <- argmax over cabType of utility[cabType]

    if bestCab is not NULL:
        CONFIRM_BOOKING(bestCab, pickup, destination)
        driver <- ASSIGN_NEAREST_DRIVER(bestCab, pickup)
        route <- COMPUTE_SHORTEST_ROUTE(pickup, destination)
        return TRACK_RIDE(driver, route)
    else:
        return "No cabs currently available – retry"
```

Note: The `UTILITY_FUNCTION` combines normalised cost, waiting time, and comfort rating (weighted according to the customer's stated preferences) into a single score, allowing the agent to rationally choose the cab option that maximises the customer's expected satisfaction.

Question 5 — Uniform Cost Search on the Delivery Network

The logistics network below is a weighted graph connecting warehouses. We must find the least-cost path from the starting warehouse *S* to the destination warehouse *G* using Uniform Cost Search (UCS).



1. Graph Edges (Directed, Weighted)

From	To	Cost
S	A	1
S	G	12
A	B	3
A	C	1
B	D	3
C	D	1
C	G	2
D	G	3

2. UCS Algorithm — Method

Uniform Cost Search expands the frontier node with the *lowest cumulative path cost* $g(n)$ from the start node, using a priority queue. A node is only goal-tested when it is popped (removed) from the frontier, since UCS guarantees that the first time a node is popped, the optimal (minimum-cost) path to it has been found.

3. Step-by-Step Trace

Iter.	Node Expanded (g)	Edges Relaxed	Frontier After
1	S (0)	$S \rightarrow A : 1 \cdot S \rightarrow G : 12$	A(1), G(12)
2	A (1)	$A \rightarrow B : 1+3=4 \cdot A \rightarrow C : 1+1=2$	C(2), B(4), G(12)
3	C (2)	$C \rightarrow D : 2+1=3 \cdot C \rightarrow G : 2+2=4$ (replaces G(12))	D(3), B(4), G(4)
4	D (3)	$D \rightarrow G : 3+3=6$ ($6 > 4$, G(4) not updated)	B(4), G(4)
5	B (4)	$B \rightarrow D : 4+3=7$ (D already expanded at cost 3, ignored)	G(4)
6	G (4)	Goal popped — lowest cost in frontier — search stops	—

4. Optimal Path and Cost

Tracing parent pointers back from G: $G \leftarrow C \leftarrow A \leftarrow S$.

Optimal path: $S \rightarrow A \rightarrow C \rightarrow G$

Total cost: $1 + 1 + 2 = 4$

5. Verification Against Other Possible Paths

Path	Cost Calculation	Total Cost
$S \rightarrow A \rightarrow C \rightarrow G$	$1 + 1 + 2$	4 ✓ (optimal)

Path	Cost Calculation	Total Cost
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1 + 1 + 1 + 3$	6
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1 + 3 + 3 + 3$	10
$S \rightarrow G$ (direct)	12	12

6. Interpretation of Result

Uniform Cost Search correctly avoids being misled by the direct edge $S \rightarrow G$ (cost 12), which looks tempting but is far from optimal. Instead, it systematically expands nodes in increasing order of cumulative cost and finds that routing the package through warehouses **A and C** costs only **4 units**, the minimum possible cost from S to G — confirming UCS's guarantee of finding the least-cost path in a graph with non-negative edge weights.